

ONLINE QUIZZ APP

Introduction:

Quiz App is a versatile and user-friendly web application designed to streamline the creation, management, and delivery of online quizzes and assessments. Suitable for academic evaluations, training tests, and knowledge checks, it offers a comprehensive set of features — including timed quiz creation, multiple-choice questions, and performance tracking — all accessible through a clean, role-based interface. Users interact with the platform according to their assigned role: Administrators oversee the entire system, Lecturers author and publish quizzes, and Students take assessments and review their results.

Built on the MERN stack (MongoDB, Express.js, React.js, Node.js), Quiz App incorporates robust security features such as JWT authentication, bcrypt password hashing, rate limiting, and role-based access control to ensure data integrity and appropriate access at every level. With built-in performance analytics, Lecturers can monitor attempt counts and review individual scores, while students can track their quiz history for self-improvement. Its flexibility, ease of use, and powerful role-driven features make it an ideal choice for universities, educators, and organizations seeking an efficient and secure online assessment solution.

SCENARIO Based Case Study:

Meet Priya, a university lecturer with a packed teaching schedule who values efficiency and organization in her work. Priya is responsible for conducting regular assessments for her students, which requires her to prepare question sets, evaluate performance, and track progress across multiple subjects and batches.

Priya discovers the Quiz App, a comprehensive web application designed to streamline online assessments and quiz management. Intrigued by its features, she decides to explore how the Quiz App can help her enhance her productivity and effectiveness.

User Registration and Authentication: Priya registers a Lecturer account on the Quiz App, providing her details and credentials. Upon Admin approval, she logs in securely using her credentials, ensuring that her data and quiz content remain private and protected.

Quiz Creation and Management: Priya explores the Quiz App's intuitive quiz builder, which allows her to create quizzes with a title, description, category, and time limit. She adds multiple-choice questions with four answer options each, marks the correct answers, and publishes the quiz to make it available to her students.

Student Assessment: Once a quiz is published, students can browse and attempt it within the set time limit. The platform enforces a one-attempt-per-quiz rule, ensuring fair and authentic assessments without the possibility of repeated attempts.

Performance Analytics: The Quiz App provides Priya with tools to monitor quiz performance. She can view the total number of attempts, review individual student scores, and identify areas where students may need additional support, helping her make informed teaching decisions.

Student Experience: Students can browse all available quizzes, take timed assessments, and instantly view their results and score breakdowns after submission. They can also revisit their attempt history to track their own progress over time.

Priya's Experience: Thanks to the Quiz App, Priya can now conduct and manage assessments more efficiently and effectively. She can focus on teaching and mentoring her students, confident that the platform handles quiz delivery, timing, and evaluation seamlessly. The Quiz App has become an indispensable tool in her academic workflow, helping her achieve her professional goals with ease and confidence.

SYSTEM REQUIREMENTS:

Software Requirements:

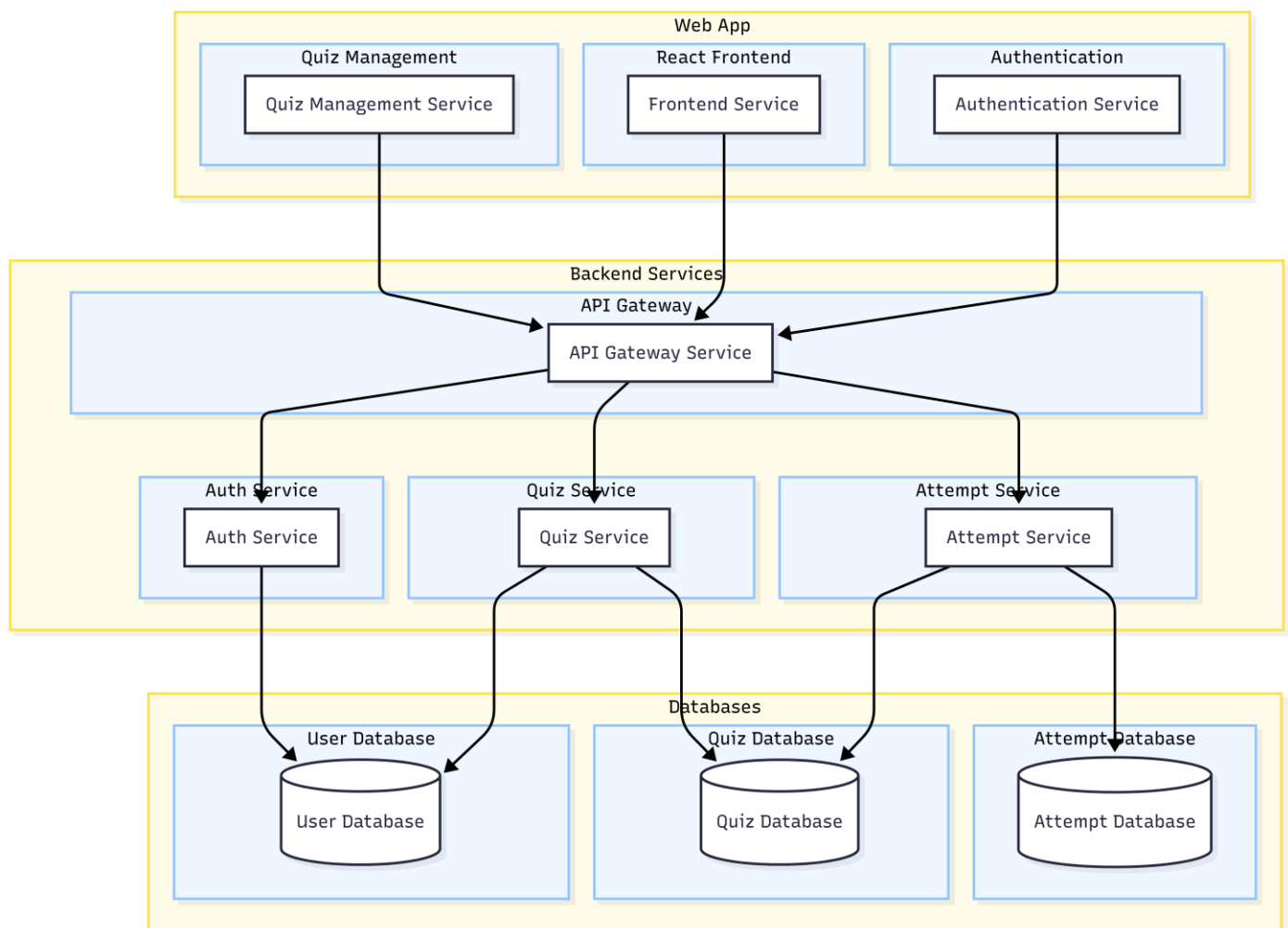
Component	Requirement	Purpose
Operating System	Windows 10/11, macOS, Linux	Cross-platform dev & testing
Node.js	v16 or above	Runtime for backend & frontend tooling
npm	v8 or above	Package manager for dependencies
React.js	v18+	Frontend UI library
Express.js	v4.21+	RESTful API framework
MongoDB	v6+	NoSQL database for all application data
Vite	v5+	Frontend build tool and dev server
Browser	Chrome / Firefox (latest)	Frontend rendering and testing
Postman	Any recent version	API testing during development
VS Code	Any recent version	Recommended code editor

Hardware Requirements:

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 (8th Gen+) / AMD Ryzen 5
RAM	4 GB	8 GB or higher
Storage	500 MB free space	1 GB+ for packages and DB
Display	1280×720	1920×1080 or higher

PROJECT ARCHITECTURE:

TECHNICAL ARCHITECTURE:

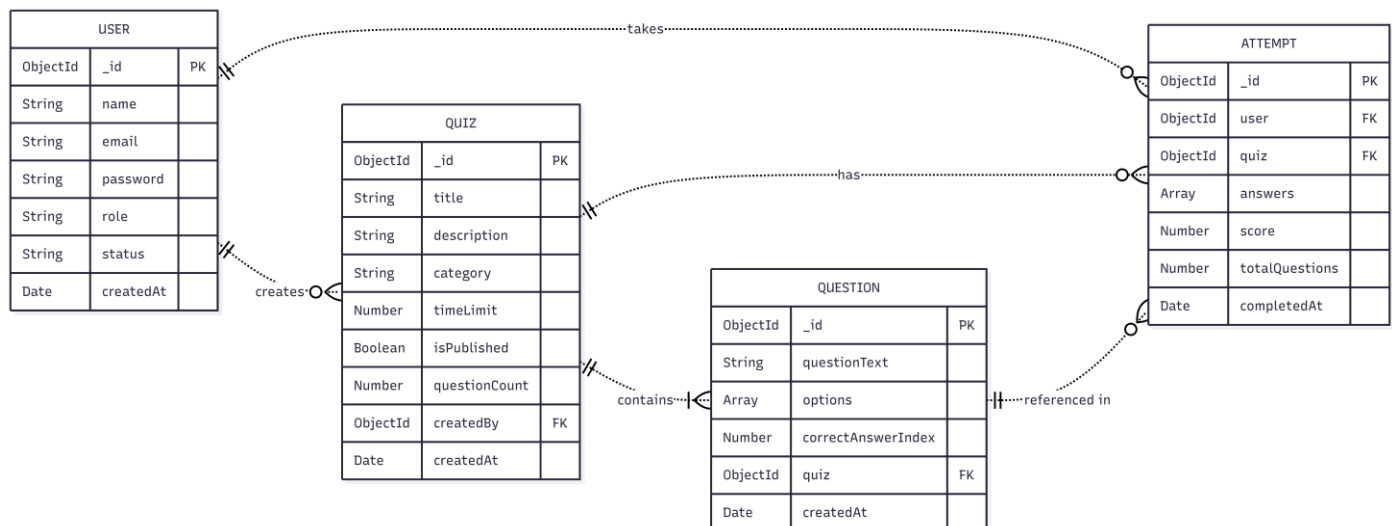


In this architecture, the flow begins with a user (represented by the "User" node) interacting with the Quiz App frontend (represented by the "React Frontend" node) built with React.js and Tailwind CSS, running on port 5173. The user performs actions such as logging in, browsing quizzes, or submitting attempts through the frontend interface.

The React frontend then sends an HTTP request — along with a Bearer JWT token in the Authorization header — to the Quiz App backend (represented by the "Express Backend" node) running on port 5000. The backend first passes the request through security middleware, including Helmet for HTTP headers, CORS validation, and rate limiting, before reaching the authentication middleware which verifies the JWT token and enforces role-based access control.

The backend communicates with a database (represented by the "MongoDB Database" node) via Mongoose ODM to perform CRUD operations — storing and retrieving users, quizzes, questions, and attempt records securely. Once the backend has processed the request and interacted with the database, it sends a structured JSON response back to the React frontend. The frontend then updates the UI state accordingly, displaying quizzes, results, dashboards, or error messages to the user depending on the response.

ER Diagram



The Entity-Relationship (ER) diagram for the Quiz App represents the various entities and their relationships within the online assessment management system. Here is a description of the key components of the ER diagram:

1. Entities:

- **User:** Represents all registered users in the system, with attributes such as user ID, name, email, password, role (user/lecturer/admin), and status (pending/approved/rejected).
- **Quiz:** Represents quizzes created by lecturers, with attributes such as quiz ID, title, description, category, time limit, isPublished flag, and question count.
- **Question:** Represents individual questions belonging to a quiz, with attributes such as question ID, question text, options (4 choices), and correct answer index.
- **Attempt:** Represents a student's quiz submission, with attributes such as attempt ID, score, total questions, answers array, and completion date.

2. Relationships:

- **User–Quiz (Creates):** A Lecturer/Admin creates one or many quizzes. Each quiz belongs to exactly one creator.
- **Quiz–Question (Contains):** A quiz contains one or many questions. Each question belongs to exactly one quiz.
- **User–Attempt (Takes):** A user submits one attempt per quiz. Each attempt is linked to exactly one user.
- **Quiz–Attempt (Has):** A quiz can have many attempts across different users. Each attempt is linked to exactly one quiz.
- **Question–Attempt (References):** Each attempt stores answers that reference individual questions.

3. Attributes: Each entity has its own set of attributes describing its properties. For example, the Quiz entity has attributes such as quiz ID, title, category, time limit, and publish status.

4. Primary Keys: Each entity has a primary key that uniquely identifies each record. MongoDB auto-generates `_id` (ObjectId) as the primary key for every document.

5. Foreign Keys: References are used to establish relationships between entities. For example, the Quiz entity stores a `createdBy` field referencing the User ID, and the Attempt entity stores both user and quiz fields as foreign references.

Overall, the ER diagram for the Quiz App provides a visual representation of the database schema, showing how users, quizzes, questions, and attempts are interrelated within the assessment management system.

Key Features:

The key features of the Quiz App can vary depending on its specific functionality and target audience. However, here are the core key features that the Quiz App includes:

User Registration and Profiles: Allow users to create accounts with their name, email, and password, and select their role (Student or Lecturer) during registration. Lecturers undergo an Admin approval process before gaining platform access, ensuring content quality and system security.

Role-Based Access Control: The platform supports three distinct roles — Admin, Lecturer, and Student — each with clearly defined permissions. Admins manage the entire system, Lecturers create and manage quizzes, and Students browse and attempt published quizzes, ensuring every user only accesses what they are authorized to.

Quiz Creation and Management: Allow Lecturers and Admins to create quizzes by providing a title, description, category, and time limit. Quizzes can be saved as drafts or published when ready, giving creators full control over content visibility.

Question Management: Enable Lecturers to add, edit, and delete multiple-choice questions within a quiz, each containing exactly four answer options and a designated correct answer, ensuring structured and consistent assessments.

Timed Assessments: Students take quizzes within a set time limit, with a live countdown timer displayed during the attempt. The platform enforces a strict one-attempt-per-quiz rule, maintaining the integrity and fairness of every assessment.

Results and Performance Tracking: Instantly display scores and detailed per-question breakdowns to students upon quiz submission, helping them identify strengths and areas for improvement. Students can also revisit their complete attempt history at any time.

Analytics and Reporting: Provide Lecturers and Admins with tools to monitor quiz performance. They can view total attempt counts per quiz and review individual student scores, helping them make informed decisions to improve content and teaching strategies.

Security and Data Protection: The platform incorporates JWT authentication, bcrypt password hashing, rate limiting on login endpoints, Helmet.js security headers, and CORS protection, ensuring user data and assessment content remain private and secure at all times.

ROLES AND RESPONSIBILITIES:

User (Student):

- **Registration and Account Management:** Students are responsible for creating an account, providing accurate personal information such as name and email, and managing their account details including password security. They select the Student role during registration and gain immediate access upon successful sign-up.
- **Quiz Discovery and Attempt:** Students can browse all published quizzes on their dashboard, filtering by category or searching by title based on their preferences. They are responsible for selecting the desired quiz and attempting it within the given time limit, ensuring they complete the assessment sincerely.
- **Assessment Integrity:** Students are responsible for attempting each quiz only once, as the platform enforces a strict one-attempt-per-quiz rule. They should engage with the assessment honestly and submit their responses within the allotted time.
- **Performance Tracking:** Students can review their results immediately after submission, including their score and per-question breakdown. They can also visit their attempt history to monitor their progress over time and identify areas for improvement.
- **Feedback:** Students have the responsibility to engage genuinely with assessments, as their attempt data contributes to the Lecturer's performance analytics and helps improve future quiz content.

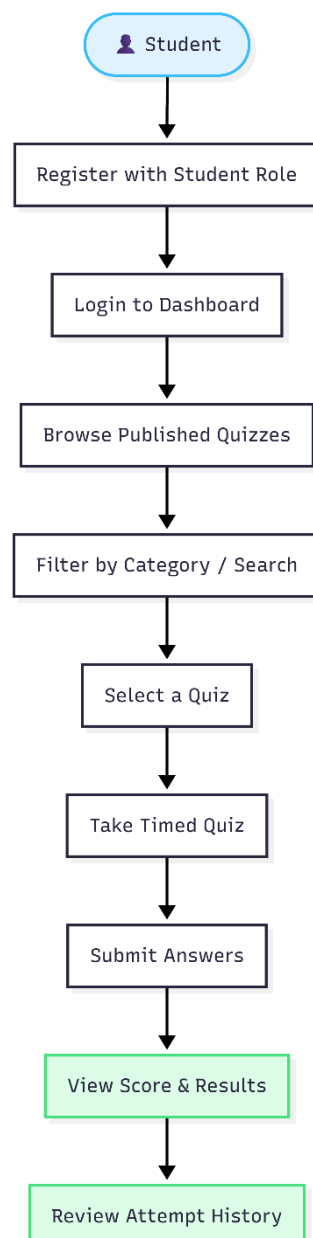
Lecturer:

- **Account Registration and Approval:** Lecturers are responsible for registering with accurate professional details and selecting the Lecturer role. Their account remains pending until approved by the Admin, after which they gain full access to quiz authoring features.
- **Quiz Creation and Management:** Lecturers are responsible for creating well-structured quizzes by providing a title, description, category, and appropriate time limit. They manage the lifecycle of their quizzes — from draft to published — and update or delete them as needed.
- **Question Authoring:** Lecturers are responsible for adding clear and accurate multiple-choice questions with exactly four answer options and correctly marking the right answer for each question, ensuring assessment quality.
- **Performance Monitoring:** Lecturers can view attempt counts and individual student scores for their quizzes. They are responsible for analyzing this data to evaluate student understanding and improve their quiz content accordingly.

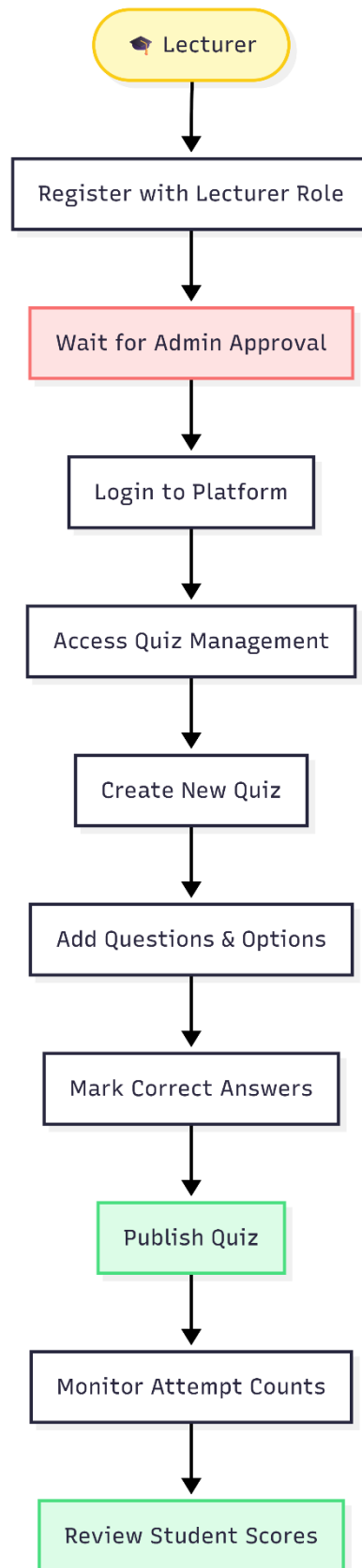
Admin:

- **System Management:** The Admin is responsible for the overall management and administration of the Quiz App. This includes monitoring platform activity, ensuring data security and privacy, and maintaining the integrity of all content across the system.
- **User Management:** The Admin is responsible for managing all user accounts, including reviewing and approving or rejecting pending Lecturer registrations. They may also suspend or delete any user account when necessary to maintain platform standards.
- **Lecturer Oversight:** The Admin oversees all Lecturers on the platform, verifying their credentials through the approval workflow before granting quiz creation access. They ensure the quality and appropriateness of all quizzes published on the platform.
- **Quiz and Content Control:** The Admin has full visibility and control over all quizzes regardless of the creator. They can edit or delete any quiz or question on the platform to ensure content accuracy and compliance with institutional standards.

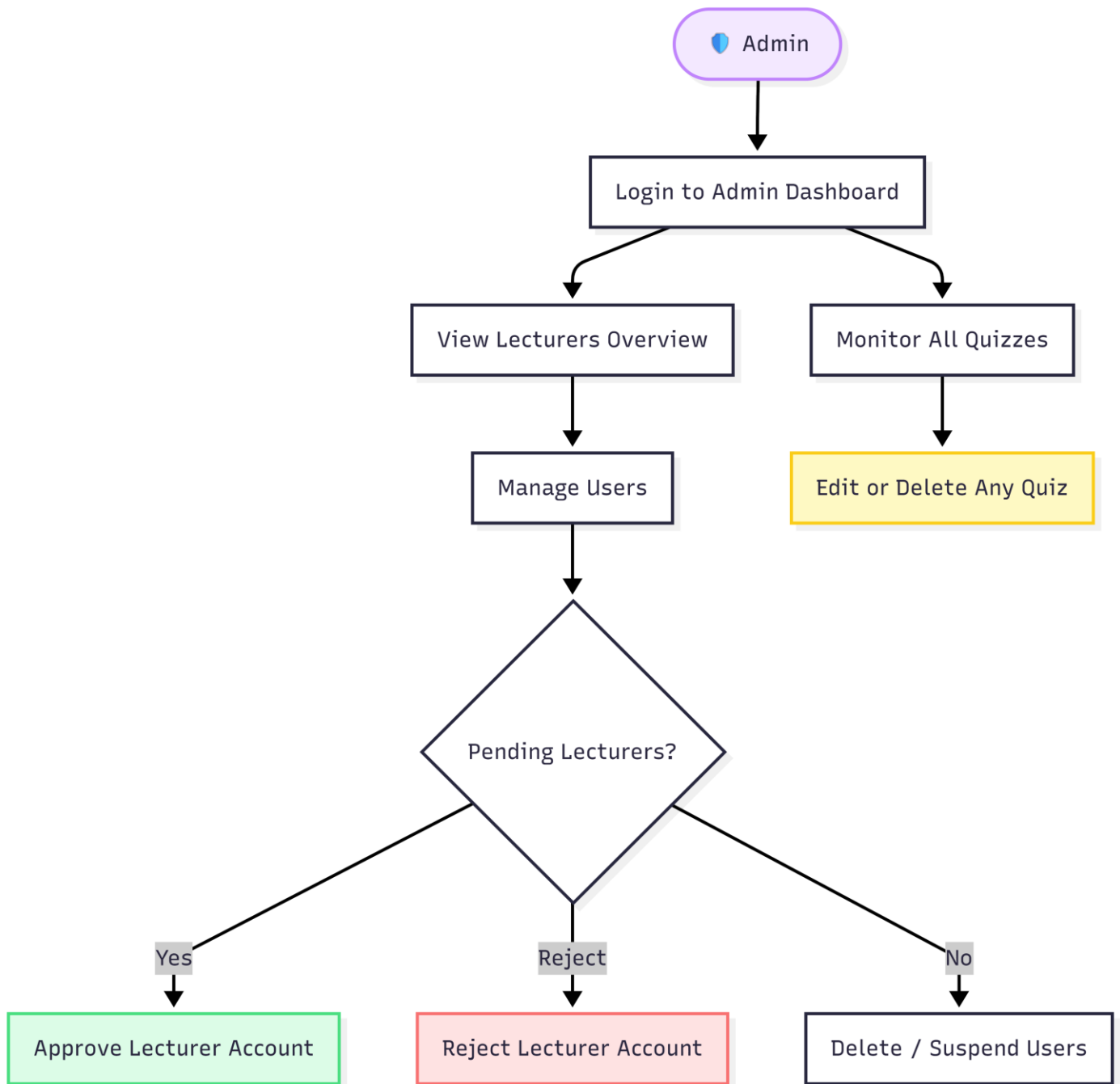
User Flow:



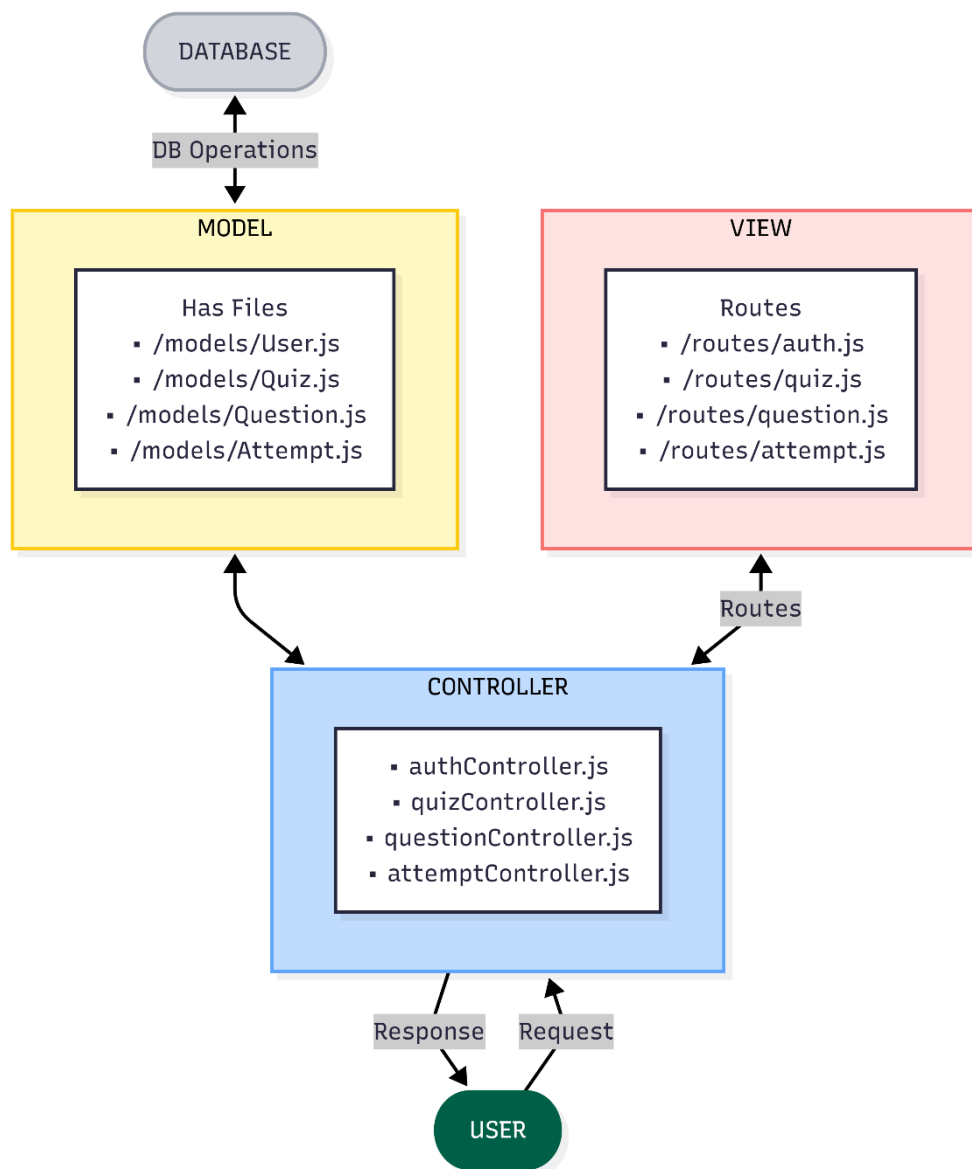
Lecturer Flow:



Admin Flow:



MVC Pattern:



The Quiz App application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic in the Quiz App. This includes the definition of data schemas and the operations performed on the MongoDB database using those schemas. The models are implemented using Mongoose, which provides a schema-based solution to model application data. The Quiz App defines four core models — User, Quiz, Question, and Attempt — each representing a key entity in the system. The User model also includes a pre-save middleware hook for password hashing and an instance method for password comparison during authentication.

Controller Layer

The Controller layer acts as an intermediary between the view (routes) and the model in the Quiz App. It receives incoming HTTP requests, processes the input (which may include JWT validation, role verification, or data transformation), calls the appropriate methods from the relevant model, and then returns a structured JSON response to the client. For example, the `quizController` handles business logic such as filtering quizzes by role, enforcing the one-attempt-per-quiz rule, and cascading deletions across related collections when a quiz is removed.

View Layer (Routing Layer)

In the context of the Quiz App's backend REST API, the View is implemented as the routing layer, where various API endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions through the relevant middleware chain. For example, a POST request to `/api/quizzes` first passes through the `protect` middleware for JWT verification, then through the `lecturerOrAdmin` middleware for role validation, before finally reaching the `createQuiz` controller function.

Advantages of Using MVC in This Project

- **Separation of Concerns:** Each layer has a clearly defined responsibility — models handle data, controllers handle logic, and routes handle request mapping — improving readability and maintainability.
- **Scalability:** New features such as additional quiz types or user roles can be added easily by creating new routes, controllers, and models without disrupting existing code.
- **Reusability:** Middleware functions like `protect`, `adminOnly`, and `lecturerOrAdmin` are reused across multiple routes, eliminating code duplication.
- **Testability:** Each layer can be tested independently — controllers and models can be unit tested without needing the full application to be running.
- **Collaboration-Friendly:** Multiple developers can work simultaneously on different layers, for example one on models and another on controllers, without merge conflicts.

Project Setup and Configuration:

Creating the Project Folder

1. Create a new folder with the name **Quiz App**.
 2. Inside that folder, create two new folders.
 3. Name one as **frontend**.
 4. Name the other as **backend**.
 5. Open the project folder in VS Code.
-

Client Setup (Installing React App)

Open the **frontend** folder in the terminal of VS Code and run the following commands:

Create the Vite + React app:

```
npm create vite@latest . -- --template react
```

Select React framework from the given options:

Select a framework:

| React

Select JavaScript variant:

Select a variant:

| JavaScript

Install all packages:

```
npm install
```

Install additional dependencies:

```
npm install react-router-dom axios react-hot-toast
```

Install Tailwind CSS:

```
npm install -D tailwindcss postcss autoprefixer
```

```
npx tailwindcss init -p
```

Start the React development server:

```
npm run dev
```

The frontend runs on <http://localhost:5173>

Server Setup (Backend)

Open the **backend** folder in the terminal of VS Code and run the following commands:

Initialize Node.js project:

```
npm init -y
```

Install all backend dependencies:

```
npm install express mongoose dotenv bcryptjs jsonwebtoken cors helmet express-rate-limit express-validator
```

Install development dependency:

npm install -D nodemon

Create the following files and folders:

Files:

- server.js
- .env
- Folders:
 - config/
 - controllers/
 - middleware/
 - models/
 - routes/

Start the backend development server:

npm run dev

The backend runs on <http://localhost:5000>

Environment Configuration (.env)

Create a .env file in the backend root and add the following:

PORT=5000

MONGO_URI=mongodb://localhost:27017/quizapp

JWT_SECRET=your_strong_secret_key_here

JWT_EXPIRES_IN=7d

package.json Scripts

Add the following scripts to your backend package.json:

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}
```

Running the Full Application

1. Open two terminals in VS Code.

2. In the first terminal, navigate to the **backend** folder and run:

`npm run dev`

3. In the second terminal, navigate to the **frontend** folder and run:

`npm run dev`

4. Open your browser and visit <http://localhost:5173> to access the Quiz App.